

1. Print Process ID (PID) and Parent Process ID (PPID) of a Process.....	1
2. Create N Child Processes.....	1
3. Parent Creates One Child and Child Creates Another.....	1
4. Process Replacement Using exec.....	2
5. Parent Waits for Child Processes to Exit.....	3
6. Parent Creates Two Child Processes, One Replaces Itself Using execl.....	3
7. Use fork() to Create a Child and Print Parent-Child Relationship.....	4
8. Forking a Process and Replace it with a Program.....	4
9. Creating Multiple Processes and Parent Waiting for All.....	5
10. Parent Creates a Child, Child Replaces Itself, and Parent Waits.....	5
11. Fork and Print Process ID and Parent Process ID for Multiple Children.....	6
12. Parent Waits for Specific Child Process.....	7
13. Create a Child, Replace It with a Program, and Parent Waits.....	7
14. Parent Creates Child, Child Creates Another, and Parent Waits for Both.....	8
15. Fork a Child and Replace It with date Command.....	9
16. Parent Creates Multiple Children and Each Child Executes a Different Command.....	9
17. Fork Child Processes and Print Parent-Child Relations.....	10
18. Child Replaces Itself with ps Command and Parent Waits.....	10
19. Parent and Child Processes with Shared Memory.....	11
20. Parent and Child Using waitpid() for Synchronized Execution.....	12
21. Parent Creates Two Children, Each Performs Different Tasks.....	12
22. Parent Creates Multiple Child Processes and Uses wait() for Synchronization.....	13
23. Create Child Process, Replace It with cat Command, and Parent Waits.....	14
24. Parent Creates Child, and Each Child Executes Different Commands.....	14
25. Parent Process Creates Child, Child Replaces Itself with pwd, Parent Waits.....	15
26. Fork Child, Replace with top, Parent Waits for Child.....	16
27. Fork Multiple Child Processes, Each Replaces Itself with date.....	16
28. Parent Creates Child, and Child Creates Another Child.....	17
29. Parent and Child Communicate via Pipe.....	18
30. Parent Creates Child, Child Creates Another Child, and Parent Waits for Both.....	18
31. Parent Process Creates Child, Child Terminates with a Specific Exit Code.....	19

Here are a few solved System Programming questions that involve process management, memory allocation, and process creation/termination in Linux C programming, similar to the one you've requested:

1. Print Process ID (PID) and Parent Process ID (PPID) of a Process

Write a program that prints the PID and PPID of the process.

Solution:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());
    return 0;
}
```

2. Create N Child Processes

Create `n` child processes, each child will print its own PID and the PID of its parent process (PPID).

Solution:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int n = 5; // number of child processes
    pid_t pid;

    for (int i = 0; i < n; i++) {
        pid = fork();
        if (pid == 0) {
            printf("Child Process: PID = %d, PPID = %d\n", getpid(), getppid());
            return 0; // Child process exits
        }
    }

    return 0;
}
```

3. Parent Creates One Child and Child Creates Another

The parent creates one child process, which in turn creates another child process. Print the PID and PPID of each process.

Solution:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) { // First child
        printf("First Child: PID = %d, PPID = %d\n", getpid(), getppid());
        pid2 = fork();
        if (pid2 == 0) { // Second child
            printf("Second Child: PID = %d, PPID = %d\n", getpid(), getppid());
        }
        return 0;
    }
    // Parent
    printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

4. Process Replacement Using **exec**

Write a program where the parent creates a child, and the child process is replaced by a new program using **exec()**.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child process before exec: PID = %d, PPID = %d\n", getpid(), getppid());
        execlp("/bin/ls", "ls", NULL); // Replaces the child process with the "ls" command
        // If exec fails
        perror("execlp failed");
        exit(1);
    } else {
        // Parent process
        printf("Parent process: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}

```

5. Parent Waits for Child Processes to Exit

Parent process creates a child process, waits for it to finish using `wait()`, and then exits.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    } else {
        // Parent process
        wait(NULL); // Wait for the child process to finish
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}
```

6. Parent Creates Two Child Processes, One Replaces Itself Using `exec1`

The parent process creates two child processes, one of which replaces itself using `exec1`.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        // First child process
        printf("First Child: PID = %d, PPID = %d\n", getpid(), getppid());
        execlp("/bin/l", "l", NULL); // Replace with "l"
    }
```

```

    } else {
        pid2 = fork();
        if (pid2 == 0) {
            // Second child process
            printf("Second Child: PID = %d, PPID = %d\n", getpid(), getppid());
            return 0;
        }
    }
    // Parent process
    wait(NULL); // Wait for both child processes
    wait(NULL);
    printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

7. Use `fork()` to Create a Child and Print Parent-Child Relationship

Fork a process and print the PID of both parent and child.

Solution:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());
    } else {
        // Parent process
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}

```

8. Forking a Process and Replace it with a Program

Create a program that forks a child and replaces it with the `date` command using `execvp()`.

Solution:

```

#include <stdio.h>
#include <unistd.h>

```

```
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execlp("/bin/date", "date", NULL);
    }
    return 0;
}
```

9. Creating Multiple Processes and Parent Waiting for All

Create multiple processes and have the parent process wait for each of them to finish.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 3; // Create 3 child processes
    pid_t pid;

    for (int i = 0; i < n; i++) {
        pid = fork();
        if (pid == 0) {
            printf("Child %d: PID = %d, PPID = %d\n", i + 1, getpid(), getppid());
            return 0;
        }
    }

    // Parent waits for all child processes
    for (int i = 0; i < n; i++) {
        wait(NULL);
    }
    printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}
```

10. Parent Creates a Child, Child Replaces Itself, and Parent Waits

Parent creates a child process, the child replaces itself with `ls`, and the parent waits for the child to complete.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execlp("/bin/ls", "ls", NULL); // Replace child with "ls" command
    } else {
        // Parent process
        wait(NULL); // Wait for the child to complete
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}
```

Would you like more examples or explanations for any of these topics?

Here are more system programming examples that involve process management, memory allocation, process creation/termination, and process replacement. Each example demonstrates different concepts and operations that can be performed in Linux C programming:

11. Fork and Print Process ID and Parent Process ID for Multiple Children

Create `n` child processes, and each child prints its PID and PPID.

Solution:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int n = 4; // Number of child processes
    pid_t pid;

    for (int i = 0; i < n; i++) {
        pid = fork();
        if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d, PPID = %d\n", i + 1, getpid(), getppid());
            return 0; // Exit after printing
        }
    }
}
```

```

    }

    return 0;
}

```

12. Parent Waits for Specific Child Process

Create multiple children and have the parent process wait for one specific child to finish using `waitpid()`.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        // First child process
        printf("First Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    }

    pid2 = fork();
    if (pid2 == 0) {
        // Second child process
        printf("Second Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    }

    // Parent process waits for first child only
    waitpid(pid1, NULL, 0);
    printf("Parent after wait: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

13. Create a Child, Replace It with a Program, and Parent Waits

The parent creates a child, the child replaces itself with a program using `execvp()`, and the parent waits for the child to finish.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());
        execlp("/bin/ls", "ls", "-l", NULL); // Replace with "ls -l"
    } else {
        // Parent process
        wait(NULL); // Wait for the child process to finish
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}
```

14. Parent Creates Child, Child Creates Another, and Parent Waits for Both

The parent creates a child, which in turn creates another child. The parent waits for both child processes to finish.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        // First child process
        pid2 = fork();
        if (pid2 == 0) {
            // Second child process
            printf("Second Child: PID = %d, PPID = %d\n", getpid(), getppid());
            return 0;
        }
        printf("First Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    }
}
```

```

// Parent process waits for both children
wait(NULL);
wait(NULL);
printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
return 0;
}

```

15. Fork a Child and Replace It with `date` Command

The parent creates a child process, and the child replaces itself with the `date` command using `execvp()`.

Solution:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execvp("/bin/date", "date", NULL); // Replace with the "date" command
    }
    return 0;
}

```

16. Parent Creates Multiple Children and Each Child Executes a Different Command

The parent creates multiple child processes, and each child executes a different command using `execvp()`.

Solution:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;
    char *commands[] = {"/bin/ls", "/bin/date", "/bin/ps"};
    for (int i = 0; i < 3; i++) {
        pid = fork();
        if (pid == 0) {
            // Each child executes a different command

```

```

        execlp(commands[i], commands[i], NULL);
    }
}
return 0;
}

```

17. Fork Child Processes and Print Parent-Child Relations

Create multiple child processes and have each child print its PID and PPID. The parent prints after all children finish.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int n = 3; // Number of child processes

    for (int i = 0; i < n; i++) {
        pid = fork();
        if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d, PPID = %d\n", i + 1, getpid(), getppid());
            return 0; // Exit after printing
        }
    }

    // Parent process waits for all child processes
    for (int i = 0; i < n; i++) {
        wait(NULL);
    }

    printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

18. Child Replaces Itself with **ps** Command and Parent Waits

The parent creates a child process, which replaces itself with the **ps** command using `execlp()`, and the parent waits for the child to finish.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execlp("/bin/ps", "ps", NULL); // Replace with "ps" command
    }
    wait(NULL); // Parent waits for child
    return 0;
}
```

19. Parent and Child Processes with Shared Memory

Use shared memory to store a variable that is updated by the parent and read by the child process.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/types.h>

int main() {
    int shmid;
    pid_t pid;
    int *shm;

    shmid = shmget(IPC_PRIVATE, sizeof(int), 0666);
    shm = (int *)shmat(shmid, NULL, 0);

    *shm = 0; // Initialize shared memory variable

    pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child reads shared memory: %d\n", *shm);
        return 0;
    } else {
        // Parent process
        *shm = 100; // Update shared memory
        wait(NULL); // Wait for child to finish
    }
}
```

```

    shmctl(shmid, IPC_RMID, NULL); // Clean up shared memory
    return 0;
}

```

20. Parent and Child Using `waitpid()` for Synchronized Execution

The parent creates a child and waits for it to finish using `waitpid()`, and then executes its own task.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    } else {
        // Parent process
        waitpid(pid, NULL, 0); // Wait for the child process to finish
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}

```

21. Parent Creates Two Children, Each Performs Different Tasks

Parent process creates two children, each of which performs a different task, and then waits for both children to complete.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

```

```

if (pid1 == 0) {
    // First child performs task 1
    printf("First Child: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

pid2 = fork();
if (pid2 == 0) {
    // Second child performs task 2
    printf("Second Child: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

// Parent waits for both children
wait(NULL);
wait(NULL);
printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
return 0;
}

```

These examples cover various use cases of process creation, synchronization, and replacement using `fork()`, `wait()`, `execvp()`, and shared memory. Let me know if you need further explanations or more examples on specific topics!

Here are more system programming questions similar to the ones you've seen, focusing on process management, creation, synchronization, and process replacement.

22. Parent Creates Multiple Child Processes and Uses `wait()` for Synchronization

Create `n` child processes where each child prints its PID and PPID. The parent waits for all children to finish before exiting.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int n = 5; // Number of children
    pid_t pid;

    for (int i = 0; i < n; i++) {

```

```

    pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child %d: PID = %d, PPID = %d\n", i + 1, getpid(), getppid());
        return 0; // Exit after printing
    }
}

// Parent process waits for all child processes
for (int i = 0; i < n; i++) {
    wait(NULL);
}

printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
return 0;
}

```

23. Create Child Process, Replace It with `cat` Command, and Parent Waits

The parent process creates a child, which replaces itself with the `cat` command, and the parent waits for the child.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execlp("/bin/cat", "cat", "file.txt", NULL); // Replace with "cat file.txt"
    } else {
        // Parent process
        wait(NULL); // Wait for the child to finish
    }
    return 0;
}

```

24. Parent Creates Child, and Each Child Executes Different Commands

Create `n` child processes, and each child executes a different command using `execlp()`.

Solution:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;
    char *commands[] = {"/bin/ls", "/bin/date", "/bin/pwd"};
    int n = 3; // Number of children

    for (int i = 0; i < n; i++) {
        pid = fork();
        if (pid == 0) {
            // Each child executes a different command
            execlp(commands[i], commands[i], NULL);
        }
    }

    // Parent waits for all child processes to finish
    for (int i = 0; i < n; i++) {
        wait(NULL);
    }

    return 0;
}
```

25. Parent Process Creates Child, Child Replaces Itself with `pwd`, Parent Waits

Create a child, replace it with the `pwd` command using `execlp()`, and then the parent waits for the child to finish.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        execlp("/bin/pwd", "pwd", NULL); // Replace with "pwd" command
    } else {
        // Parent process
        wait(NULL); // Wait for child to finish
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
}
```



```

    }
    return 0;
}

```

26. Fork Child, Replace with **top**, Parent Waits for Child

The parent creates a child, and the child replaces itself with the **top** command using `exec1p()`. The parent waits for the child to finish.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        exec1p("/usr/bin/top", "top", NULL); // Replace with "top"
    } else {
        // Parent process
        wait(NULL); // Wait for child to finish
        printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    return 0;
}

```

27. Fork Multiple Child Processes, Each Replaces Itself with **date**

The parent creates multiple children, and each child replaces itself with the **date** command.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int n = 3; // Number of children

    for (int i = 0; i < n; i++) {
        pid = fork();
    }
}

```

```

    if (pid == 0) {
        // Child process replaces with "date" command
        execlp("/bin/date", "date", NULL);
    }
}

// Parent waits for all child processes to finish
for (int i = 0; i < n; i++) {
    wait(NULL);
}

return 0;
}

```

28. Parent Creates Child, and Child Creates Another Child

The parent creates one child, and that child creates another child. The parent waits for both children.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        // First child creates a second child
        pid2 = fork();
        if (pid2 == 0) {
            // Second child
            printf("Second Child: PID = %d, PPID = %d\n", getpid(), getppid());
            return 0;
        }
        printf("First Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    }

    // Parent waits for both children
    wait(NULL);
    wait(NULL);
    printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

29. Parent and Child Communicate via Pipe

The parent creates a child and both processes communicate using a pipe. The parent sends data to the child, and the child reads and prints the data.

Solution:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid;
    char message[] = "Hello from parent!";
    char buffer[128];

    // Create a pipe
    pipe(pipefd);

    pid = fork();
    if (pid == 0) {
        // Child process
        close(pipefd[1]); // Close unused write end
        read(pipefd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        return 0;
    } else {
        // Parent process
        close(pipefd[0]); // Close unused read end
        write(pipefd[1], message, sizeof(message));
        wait(NULL); // Wait for child to finish
    }

    return 0;
}
```

30. Parent Creates Child, Child Creates Another Child, and Parent Waits for Both

The parent creates a child, which then creates another child. The parent waits for both to complete before exiting.

Solution:

```
#include <stdio.h>
```

```

#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 == 0) {
        // First child creates second child
        pid2 = fork();
        if (pid2 == 0) {
            // Second child
            printf("Second child: PID = %d, PPID = %d\n", getpid(), getppid());
            return 0;
        }
        printf("First child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 0;
    }

    // Parent waits for both children
    wait(NULL);
    wait(NULL);
    printf("Parent: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
}

```

31. Parent Process Creates Child, Child Terminates with a Specific Exit Code

The parent creates a child process, and the child terminates with a specific exit code. The parent retrieves the exit status of the child.

Solution:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());
        return 5; // Exit with status code 5
    } else {

```

```
// Parent process
wait(&status); // Wait for child to finish
if (WIFEXITED(status)) {
    printf("Parent: Child exited with status code %d\n", WEXITSTATUS(status));
}
}

return 0;
}
```

These questions expand upon your previous concepts and explore a variety of process management techniques. Let me know if you need further explanations or additional examples!